

Smart Calendar Project — Recommended README Structure

1 Project Overview

- Brief description of the app and its purpose
 - Example: “*Smart Calendar is an iOS app that allows users to create calendars, add events, invite collaborators, and manage permissions.*”
- Tech stack summary: Swift (frontend), Node.js (backend), Supabase (DB + Auth + RLS)
- Architecture diagram (optional but highly recommended)
- Link to Figma/UI mockups if available

2 Setup Instructions

For **any new team member** or for recovery:

- **Prerequisites:** Node.js, Xcode, Git, Supabase account
- **Environment Variables:**
 - `.env` keys for Node backend (Supabase URL, Supabase API keys, JWT secret)
 - Swift: API base URL, any config constants
- **Installation steps:**
 - Clone repo
 - Install dependencies: `npm install` for backend, Swift package setup for frontend
 - Running local server: `npm run dev`
 - Running Swift project in simulator

3 Database Setup

DB Team README section should include:

- Schema diagram (tables + relationships)
- SQL scripts for creating tables
- Example seed data
- Notes on RLS policies & permissions per table
- Any constraints or triggers that are important

4 Auth & Role Management

Auth Team README section should include:

- JWT setup and verification steps
- Role definitions (admin, collaborator, viewer, etc.)
- Middleware usage in Node APIs
- API endpoints protected by auth & role checks

5 API Documentation

API Team README section should include:

- List of endpoints with:
 - URL & HTTP method
 - Required headers (JWT)
 - Request body example
 - Response example
 - Role permissions required
- Notes on edge cases or error codes
- Postman / Insomnia collection link for testing

6 Frontend Integration Notes

Swift Team README section should include:

- API calling conventions
- How JWT tokens are stored & used in requests
- Screen-to-API mapping: which Swift view calls which endpoint
- Any mock data used during development

7 Testing Instructions

- Unit tests: how to run (Node: Jest / Swift: XCTest)
- Manual test cases: e.g., “Create calendar → Invite user → Add event → Delete calendar”
- Known bugs or limitations

8 Workflow & Collaboration Notes

- Branching strategy (feature branches, pull requests, code reviews)
- How to contribute / code style guidelines
- Rotation plan if team members switch modules
- Any daily or weekly standup notes if relevant

9 Backup & Recovery Notes

- Where to find project backups (GitHub, Supabase snapshots)
- How to restore DB locally from SQL scripts or Supabase dump
- How to regenerate `.env` keys if lost (with security instructions)

◆ Optional but Highly Recommended

- **Diagrams:** Architecture, API flow, DB relations
- **Screenshots / GIFs:** UI screens or API request/response examples
- **References:** Supabase docs, SwiftUI docs, Node.js docs
- **Contacts / Roles:** Who owns which part of the project



Best Practices for Team READMEs:

1. Use **Markdown headers** for clarity and navigation.
2. Keep **examples concrete**, like actual API request/response JSON.
3. Update **frequently** when schema, APIs, or workflows change.
4. Encourage each team member to **document even small steps**, because someone leaving should not cause a knowledge gap.